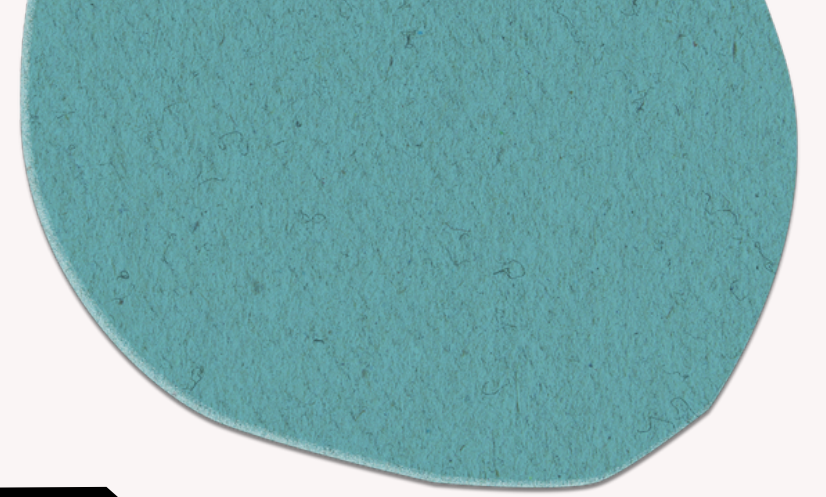


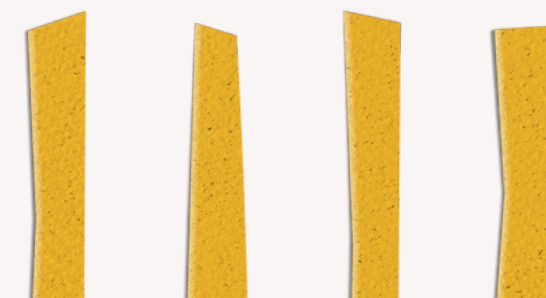


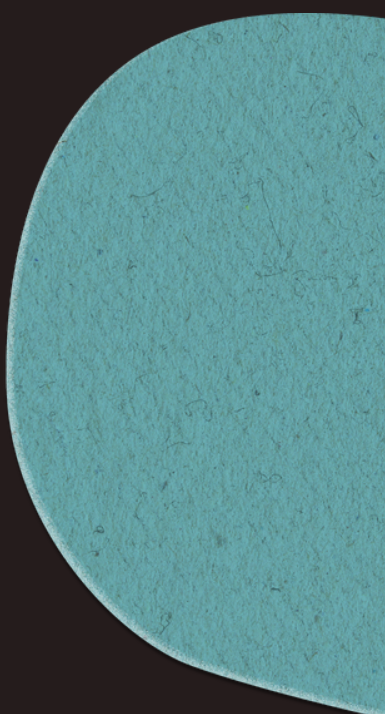
DATA STRUCTURE & ALGORITHM LAB PROJECT

MAZE SOLVER

Presented to : Miss Sahifa Hashmi

By: Umme Ruman & Rabeya Asif





DESCRIPTION

A Maze Solver is a program or algorithm designed to find a path from a starting point to an ending point in a maze.

Key Points:

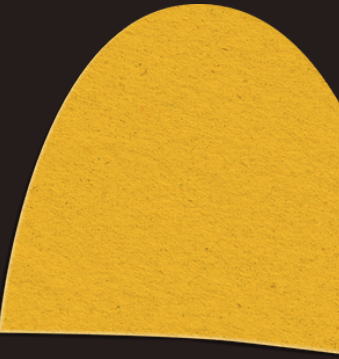
1. Input: A maze represented as a grid of cells.
 - Each cell may have walls (blocking movement) or openings.
 - Usually, a start cell and an end cell are defined.
 2. Output: A path (sequence of moves or cells) that leads from start to end.
 - The path may be the shortest path or just any valid path, depending on the algorithm.
- 

TOOL & LIBRARIES

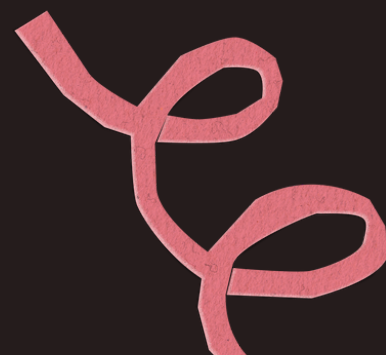
- Java Programming Language
- StdDraw → For drawing walls, paths, start & end points.
- Collections Framework:
- Stack → DFS maze generation
- Queue → BFS solving
- LinkedList → Store paths
- HashMap → Track parent cells for path reconstruction
- AWT Classes: Color, JFrame, MouseEvent, KeyEvent.



MAZE REPRESENTATION



- Maze Representation
- The maze is represented as a 2D grid of cells.
- Each cell has coordinates (x, y).
- Each cell contains 4 walls: left, right, up, down.
- Walls are removed during maze generation to create paths.
- Start and End cells are highlighted with different colors:
 - Start = Green
 - End = Red
- Visited cells are shown in grey, solution path in blue



CLASSES OVERVIEW

CELL

- Represents a single maze cell.
- Attributes:
- `int x, y` → Position of the cell
- Methods:
- `draw(r, color)` → Draws the cell as a filled circle
- `equals()` and `hashCode()` → For comparison and storage in collections

CLASSES OVERVIEW

PATH

- Stores a sequence of Cell objects.
- Uses `LinkedList<Cell>` internally.
- Can iterate through all cells using `Iterable<Cell>`.

CLASSES OVERVIEW

MAZE

- Represents the maze structure.
- Attributes:
- size → Size of maze (N x N)
- visited[][] → Marks visited cells
- left[], right[], up[], down[] → Stores walls information
- start and end → Start and end cells
- visitedPath → Path of visited cells during BFS
- solutionPath → Shortest path from start to end

METHOD

- `removeWall(x, y, direction)` → Removes walls between cells
- `isOpen(x, y, direction)` → Checks if movement is possible
- `visit(x, y)` → Marks cell as visited
- `draw()` → Draws maze walls, visited cells, and solution path

MAZEGENERATOR

- Generates a random maze using DFS backtracking.
- Starts from (0,0) and recursively visits unvisited neighbors.
- Randomly chooses directions to remove walls and create a maze.
- Sets start = (0,0) and end = (size-1, size-1).

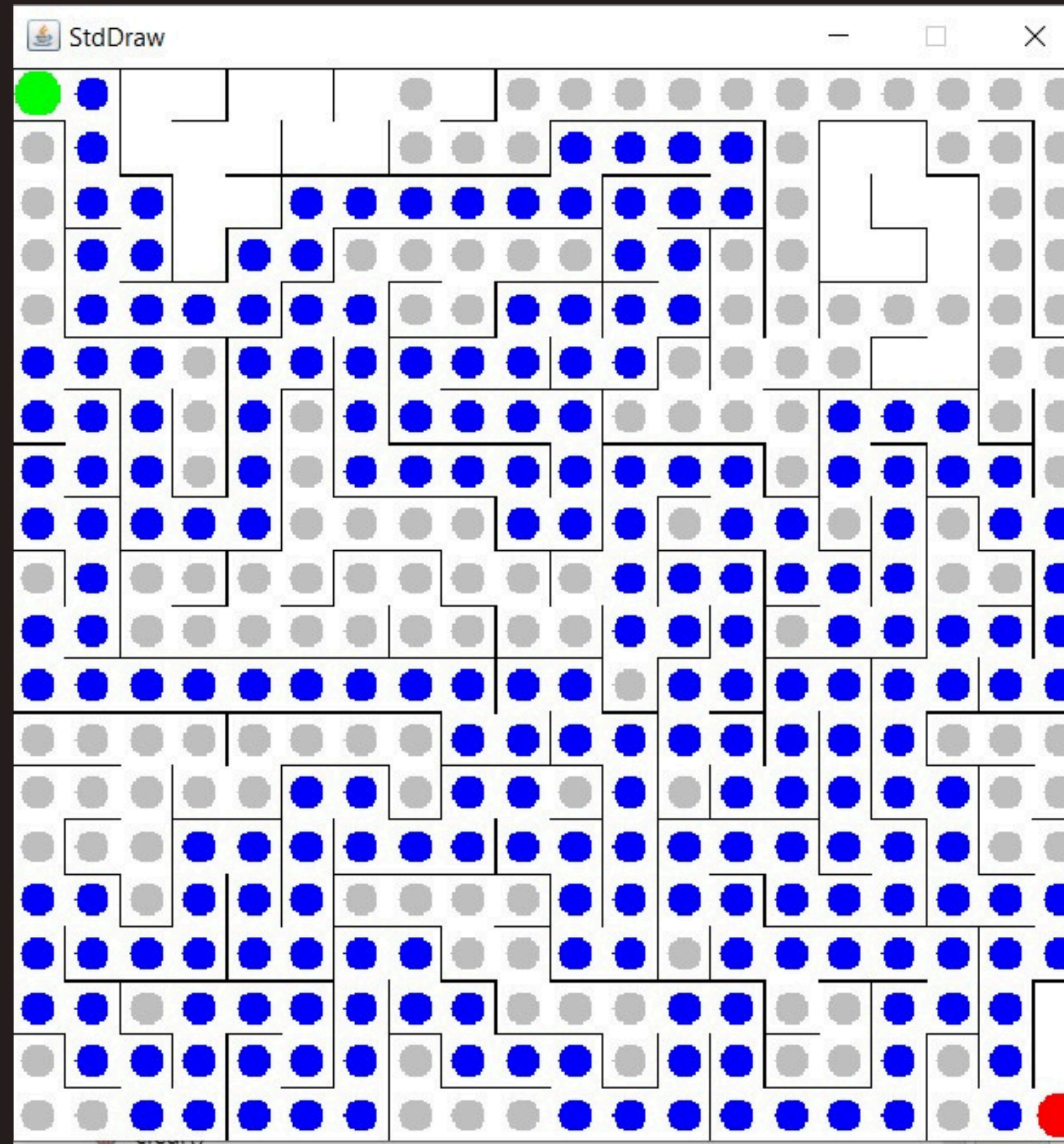
SOLVER [BFS]

- Uses Breadth-First Search to find shortest path.
- Queue stores cells to visit next.
- parent map tracks previous cell for path reconstruction.
- After BFS, the solution path is reconstructed by backtracking from end → start.

STD DRAW

- Provides a simple drawing library to visualize the maze.
- Features:
- Draw walls using `line()`
- Draw filled circles for cells using `filledCircle()`
- Mouse and keyboard input support
- Double buffering to reduce flickering

OUTPUT





MAZE GENERATION [DFS]

- DFS ensures all cells are reachable with no loops (perfect maze).
 - Algorithm:
 1. Start at initial cell (0,0)
 2. Mark current cell as visited
 3. Randomly choose an unvisited neighbor
 4. Remove wall between current cell and neighbor
 5. Move to neighbor and repeat
 6. Backtrack if no unvisited neighbors exist
- 

MAZE SOLVING [BFS]

- BFS guarantees the shortest path.
- Algorithm:
 1. Add start cell to a queue
 2. While queue is not empty:
 - Dequeue a cell
 - Explore all open neighbors that are not visited
 - Mark neighbors as visited and add them to the queue
 - Store parent information for path reconstruction
 3. Reconstruct path from end to start using parent map

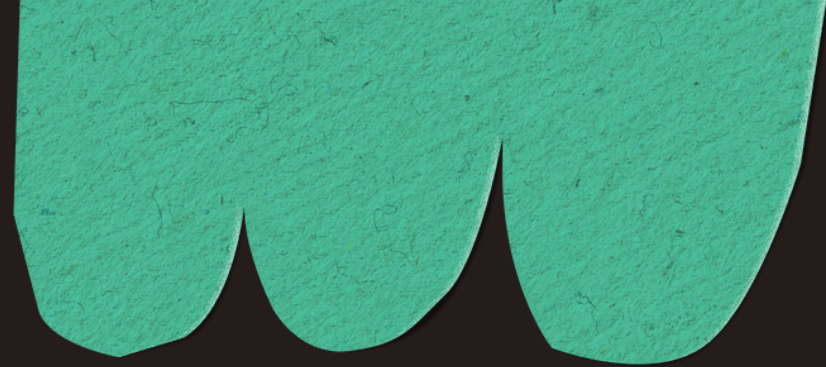
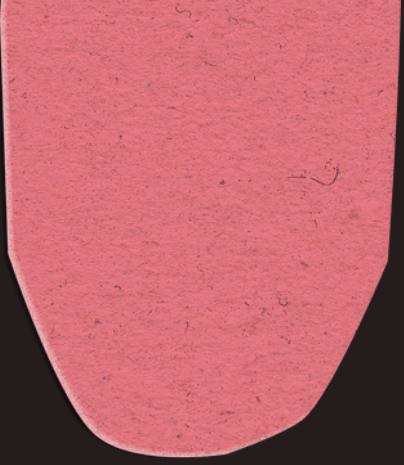


VISUALIZATION

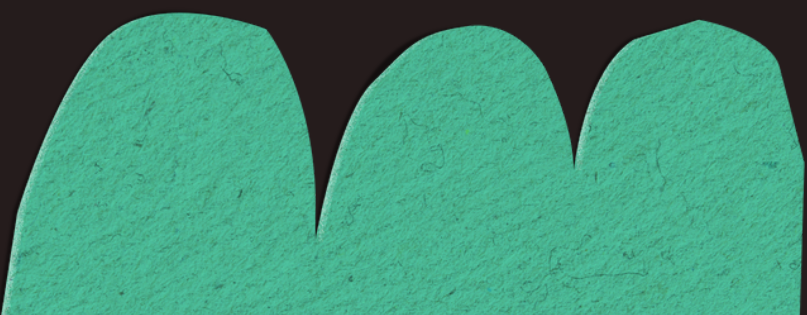
- Maze walls → Black lines
 - Visited cells → Light Grey
 - Solution path → Blue
 - Start cell → Green
 - End cell → Red
 - Updates dynamically during BFS traversal
- 

MAIN PROGRAM FLOW

- Initialize StdDraw for graphics
- Generate a maze using MazeGenerator
- Solve maze using BFS solve() method
- Draw the maze, visited path, and solution path



ALGORITHMS USED

- DFS (Depth First Search):
 - Explores as far as possible along each branch before backtracking.
 - Can be used to generate mazes or find a path (not always shortest).
 - BFS (Breadth First Search):
 - Explores all neighbors level by level.
 - Guarantees the shortest path in an unweighted maze.
- 
- 

APPLICATIONS

- Video games (NPC pathfinding)
- Robotics (finding paths in a maze)
- Puzzles and AI research
- Visualization of graph traversal algorithms



THANK YOU!

